

TRABAJO PRÁCTICO INTEGRADOR: **GESTIÓN DE DATOS DE PAÍSES EN PYTHON**

UNIVERSIDAD TECNOLÓGICA NACIONAL TECNICATURA UNIVERSITARIA EN
PROGRAMACIÓN A DISTANCIA

INTEGRANTES DEL GRUPO: Individual

Integrante: [Suarez Pablo Santiago] — [47634903]

Fecha de Entrega: [20/06/26]

Repositorio GitHub: <https://github.com/ChetoNeo/Trabajo-practico-integrador---Programación-I>

Video Demostrativo: <https://youtu.be/oRvoKEF2S34?si=Q67-GZN1iNeS2kL3>

ÍNDICE GENERAL

1. INTRODUCCIÓN Y OBJETIVOS DEL SISTEMA.....	3
2. MARCO TEÓRICO E INVESTIGACIÓN.....	3
2.1. Estructuras de Datos Lineales: Listas en Python.....	3
2.2. Estructuras de Datos Asociativas: Diccionarios.....	3
2.3. Modularización y Abstracción: Funciones.....	3
2.4. Estructuras de Control y Toma de Decisiones.....	4
2.5. Persistencia de Datos: Manipulación de Archivos CSV.....	4
2.6. Algoritmia: Técnicas de Ordenamiento y Filtrado.....	4
2.7. Análisis Estadístico de Datos.....	4
3. DISEÑO Y DECISIONES TÉCNICAS.....	4
3.1. Arquitectura del Código y Estructura de Datos Elegida.....	4
3.2. Diagrama de Flujo Principal.....	5
4. GUÍA DE USO Y CAPTURAS DE PANTALLA.....	6-7
5. DIFICULTADES ENCONTRADAS Y CONCLUSIONES.....	7
6. FUENTES BIBLIOGRÁFICAS.....	7

1. INTRODUCCIÓN Y OBJETIVOS DEL SISTEMA

El presente proyecto final integrador expone el diseño, desarrollo e implementación de una aplicación de consola en el lenguaje de programación Python (versión 3.x) destinada al procesamiento, administración y análisis de datos estadísticos correspondientes a diversos países del globo.

El sistema persigue como objetivo primordial afianzar de manera práctica e integral las competencias algorítmicas clave adquiridas a lo largo de la cursada de Programación 1. Mediante la articulación de mecanismos de lectura de almacenamiento persistente (archivos planos CSV), el modelado de datos en estructuras compuestas en memoria y la programación estructurada y modular, se provee al usuario final una interfaz interactiva de terminal capaz de realizar altas, búsquedas complejas, ordenamientos optimizados y cálculos estadísticos descriptivos rigurosos sobre el dataset provisto.

2. MARCO TEÓRICO E INVESTIGACIÓN

2.1. Estructuras de Datos Lineales: Listas en Python

Las listas en Python representan colecciones de datos ordenadas, indexadas y de carácter mutable. A diferencia de los arreglos estáticos en lenguajes de tipado estricto, las listas en Python se implementan internamente como arreglos dinámicos basados en punteros, lo que les concede la capacidad de almacenar elementos heterogéneos y redimensionar su capacidad en tiempo de ejecución de manera transparente para el programador. La lista opera como el contenedor maestro o base de datos en memoria que aloja los registros de todos los países.

2.2. Estructuras de Datos Asociativas: Diccionarios

Un diccionario es una estructura de datos no ordenada que mapea claves unívocas a valores determinados (key-value pairs). Basados en tablas de dispersión (*hash tables*), los diccionarios permiten búsquedas, inserciones y modificaciones con una complejidad temporal promedio de $O(1)$. Para este diseño, cada país se modela formalmente como un diccionario independiente, donde las claves representan las propiedades del dominio (nombre, población, superficie, continente), asegurando un acceso explícito, semántico y legible a los campos en lugar de depender de índices numéricos rígidos.

2.3. Modularización y Abstracción: Funciones

La descomposición modular mediante funciones constituye el pilar de la programación estructurada. Una función es un bloque de código organizado y reutilizable que realiza una única tarea o responsabilidad dentro del ecosistema de software. Al segmentar el sistema en subprocesos independientes (tales como la carga del archivo, el menú interactivo, o el cálculo de promedios), se incrementa la mantenibilidad del código, se reduce la duplicación de lógica (*principio DRY - Don't Repeat Yourself*) y se facilita el aislamiento de fallos durante la etapa de pruebas.

2.4. Estructuras de Control y Toma de Decisiones

El flujo de ejecución de una aplicación se gobierna mediante estructuras condicionales (if-elif-else) y lazos repetitivos (while / for). Las sentencias condicionales permiten bifurcar el comportamiento del sistema según las interacciones del usuario o la validez de los datos de entrada. Por otra parte, los ciclos while gestionan el bucle persistente de la interfaz de consola hasta recibir una instrucción de salida explícita, mientras que los ciclos for permiten iterar uniformemente las listas de países con propósitos de filtrado y visualización.

2.5. Persistencia de Datos: Manipulación de Archivos CSV

Un archivo CSV (*Comma-Separated Values*) es un formato plano estándar empleado para el intercambio de datos tabulares. Para dotar al sistema de persistencia y evitar la pérdida de información al finalizar la ejecución, se utiliza el módulo nativo csv de Python. A través del objeto csv.DictReader, el programa decodifica secuencialmente las líneas del almacenamiento secundario, procesa las cadenas de caracteres de texto y convierte de forma controlada las magnitudes numéricas a tipos de datos enteros (int) para su correcto tratamiento algebraico en memoria.

2.6. Algoritmia: Técnicas de Ordenamiento y Filtrado

El ordenamiento es el proceso de organizar un conjunto de datos en una secuencia determinada (ascendente o descendente) según un criterio específico. Python utiliza internamente el algoritmo *Timsort* (una combinación híbrida de *Merge Sort* e *Insertion Sort*), el cual ofrece una estabilidad sobresaliente y una complejidad de $O(n \log n)$ en el peor de los casos. Por su parte, el filtrado consiste en discriminar elementos basándose en predicados booleanos, aislando subconjuntos de datos que verifiquen condiciones específicas de población, superficie o pertenencia geográfica.

2.7. Análisis Estadístico de Datos

El análisis estadístico descriptivo en sistemas de software requiere la implementación de funciones agregadas sobre colecciones de datos. El cálculo de promedios aritméticos involucra sumatorias acumulativas divididas por la cardinalidad del conjunto de datos. Asimismo, la identificación de valores extremos (máximos y mínimos absolutos) exige algoritmos de búsqueda lineal mediante comparaciones iterativas, evaluando las propiedades cuantitativas de la población y superficie para extraer conclusiones analíticas del dataset.

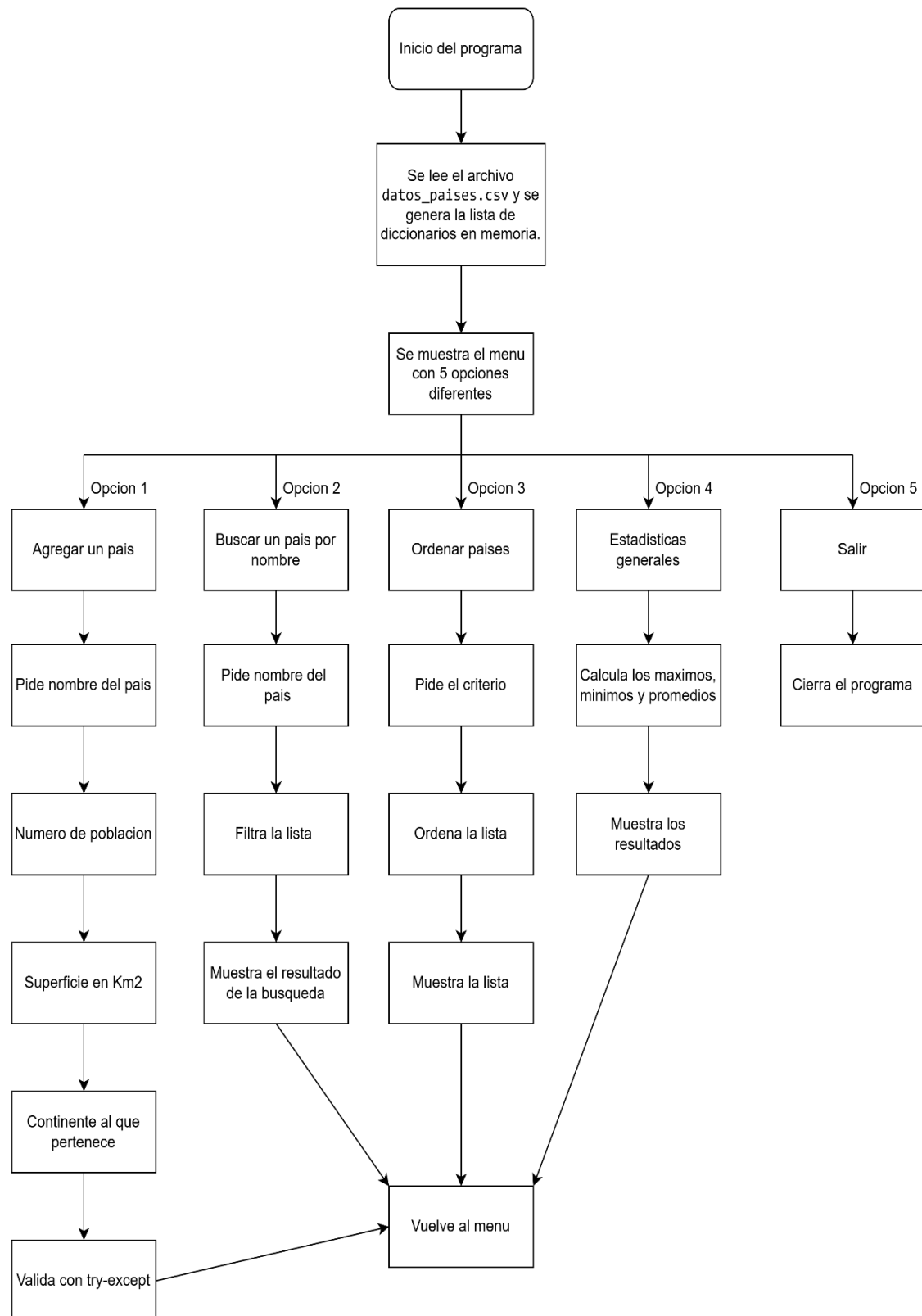
3. DISEÑO Y DECISIONES TÉCNICAS

3.1. Arquitectura del Código y Estructura de Datos Elegida

Opte por un diseño híbrido compuesto por una **lista de diccionarios**. El contenedor principal se define como una lista vacía que se puebla dinámicamente durante la inicialización del sistema. Cada celda de la lista contiene un diccionario con claves de tipo cadena y valores con tipado adecuado:

Esta decisión técnica maximiza la flexibilidad del ordenamiento nativo utilizando funciones *lambda* como claves de ordenación, garantizando un código compacto, limpio y altamente eficiente para cumplir las restricciones de la cátedra.

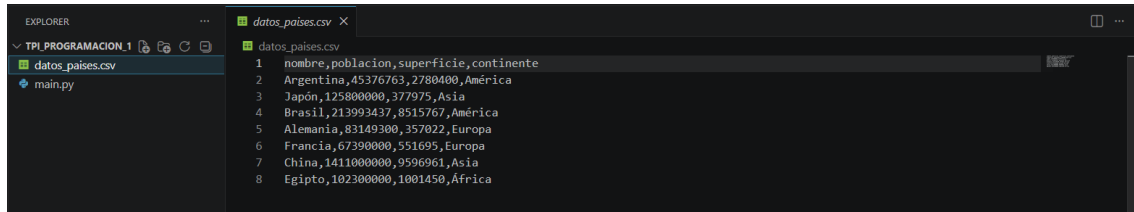
3.2. Diagrama de Flujo Principal



4. GUÍA DE USO Y CAPTURAS DE PANTALLA

El programa se inicia invocando el script main.py desde la terminal. A continuación, se adjunta la evidencia visual del correcto funcionamiento del menú y los bloques funcionales mínimos requeridos por la rúbrica.

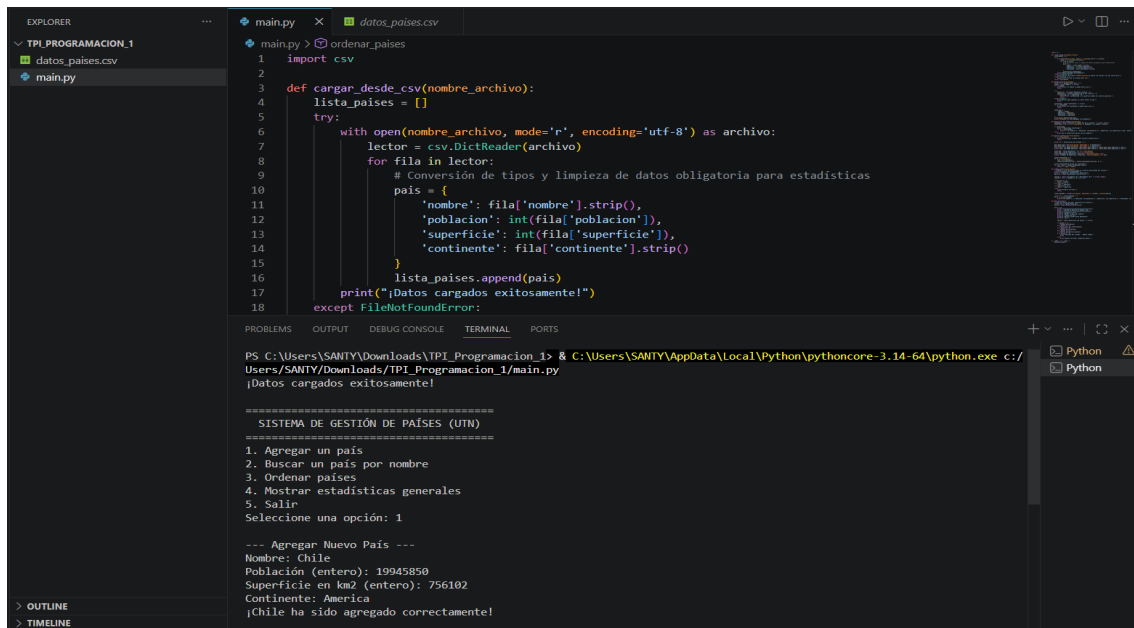
Entrada del Archivo CSV Base:



```

1 nombre,poblacion,superficie,continente
2 Argentina,45376763,2780400,América
3 Japón,125800000,377975,Asia
4 Brasil,213993437,8515767,América
5 Alemania,83149300,357022,Europa
6 Francia,67390000,551695,Europa
7 China,141100000,9596961,Asia
8 Egipto,102300000,1001450,África
  
```

Ejecución del Menú Principal y agregar un País:



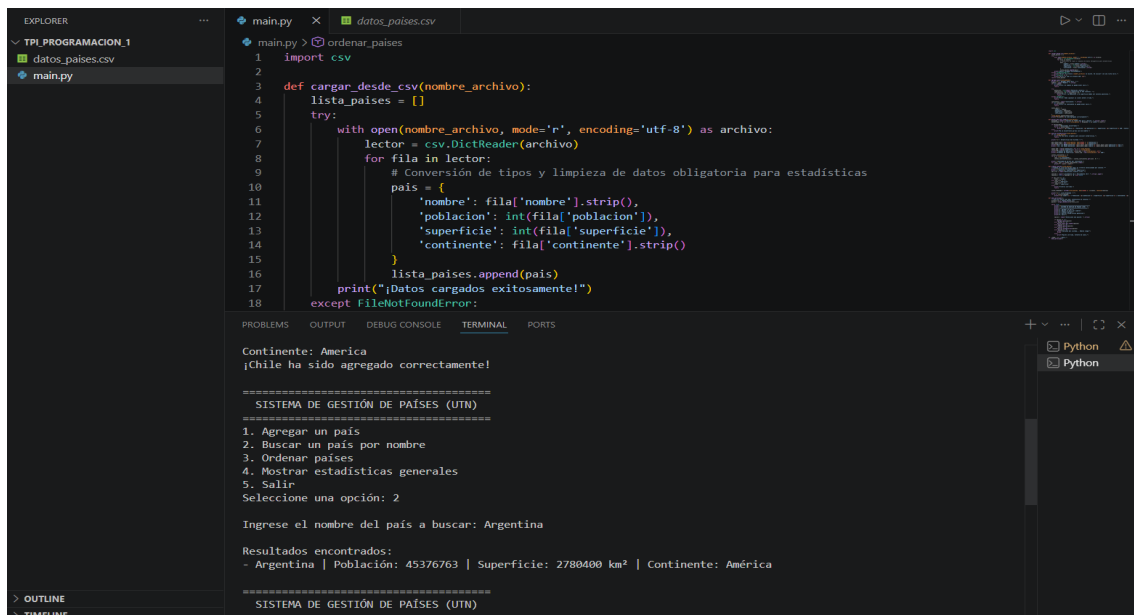
```

PS C:\Users\SANTY\Downloads\TPI_Programacion_1> & C:\Users\SANTY\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/SANTY/Downloads/TPI_Programacion_1/main.py
¡Datos cargados exitosamente!

=====
SISTEMA DE GESTIÓN DE PAÍSES (UTN)
=====
1. Agregar un país
2. Buscar un país por nombre
3. Ordenar países
4. Mostrar estadísticas generales
5. Salir
Seleccione una opción: 1

--- Agregar Nuevo País ---
Nombre: Chile
Población (entero): 19945850
Superficie en km2 (entero): 756102
Continente: America
¡Chile ha sido agregado correctamente!
  
```

Búsqueda de país:



```

Continente: America
¡Chile ha sido agregado correctamente!

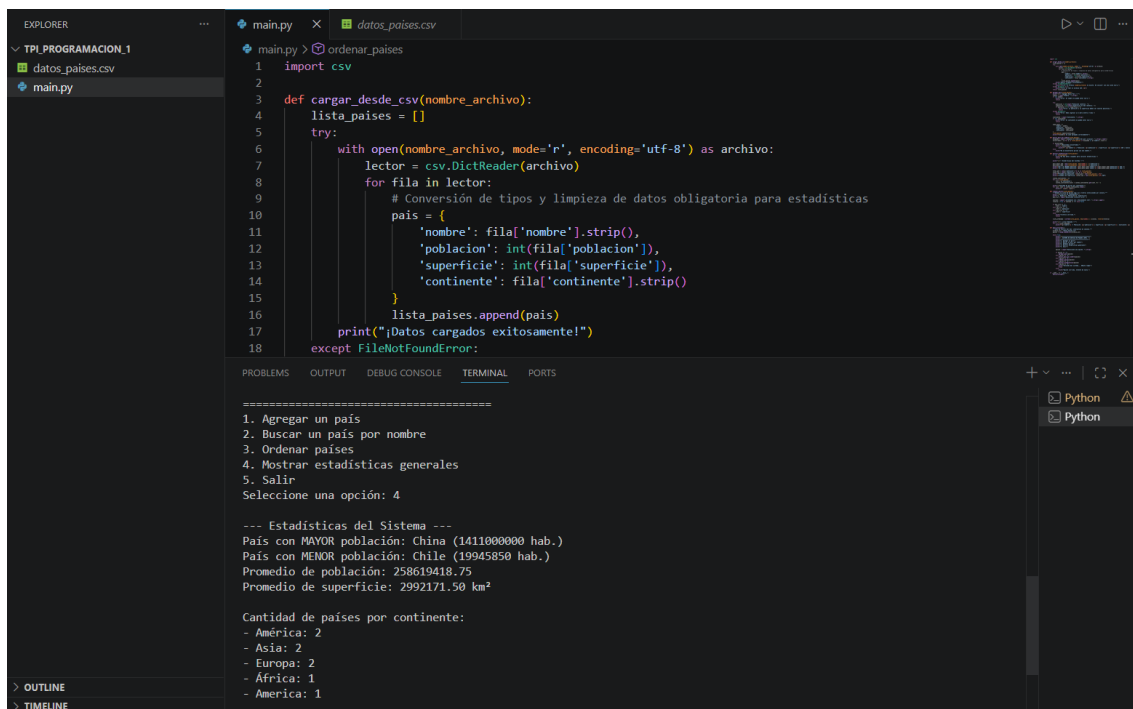
=====
SISTEMA DE GESTIÓN DE PAÍSES (UTN)
=====
1. Agregar un país
2. Buscar un país por nombre
3. Ordenar países
4. Mostrar estadísticas generales
5. Salir
Seleccione una opción: 2

Ingrese el nombre del país a buscar: Argentina

Resultados encontrados:
- Argentina | Población: 45376763 | Superficie: 2780400 km² | Continente: América

=====
SISTEMA DE GESTIÓN DE PAÍSES (UTN)
=====
  
```

Estadísticas generales:



```

EXPLORER
  TPI_PROGRAMACION_1
    datos_paises.csv
    main.py
  main.py x datos_paises.csv

main.py
1 import csv
2
3 def cargar_desde_csv(nombre_archivo):
4     lista_paises = []
5     try:
6         with open(nombre_archivo, mode='r', encoding='utf-8') as archivo:
7             lector = csv.DictReader(archivo)
8             for fila in lector:
9                 # Conversión de tipos y limpieza de datos obligatoria para estadísticas
10                pais = {
11                    'nombre': fila['nombre'].strip(),
12                    'poblacion': int(fila['poblacion']),
13                    'superficie': int(fila['superficie']),
14                    'continente': fila['continente'].strip()
15                }
16                lista_paises.append(pais)
17            print("¡Datos cargados exitosamente!")
18    except FileNotFoundError:

```

```

=====
1. Agregar un país
2. Buscar un país por nombre
3. Ordenar países
4. Mostrar estadísticas generales
5. Salir
Seleccione una opción: 4

--- Estadísticas del Sistema ---
País con MAYOR población: China (1411000000 hab.)
País con MENOR población: Chile (19945850 hab.)
Promedio de población: 258619418.75
Promedio de superficie: 2992171.50 km²

Cantidad de países por continente:
- América: 2
- Asia: 2
- Europa: 2
- África: 1
- América: 1

```

5. DIFICULTADES ENCONTRADAS Y CONCLUSIONES

El desarrollo de este TPI me permitió comprender en profundidad la importancia de la validación temprana de datos de entrada (*input sanitization*). Una de las mayores complejidades que puedo destacar es en el manejo de posibles errores en los registros numéricos provenientes del archivo CSV (como campos en blanco o caracteres inválidos). Dicha dificultad se corrigió con éxito mediante la incorporación estratégica de bloques de control de excepciones try-except y condicionales preventivos.

En conclusión, el paradigma de modularización demostró ser indispensable; aislar la lógica de cálculo estadístico de la lógica de interfaz de consola permitió un proceso de depuración ágil y un código final robusto.

6. FUENTES BIBLIOGRÁFICAS

1. **Python Software Foundation (2026).** *Python Language Reference, version 3.x: Data Structures and CSV File Reading*. Disponible en: <https://docs.python.org/3/tutorial/datastructures.html>
2. **Van Rossum, G. (2018).** *El tutorial de Python*. Manual oficial de referencia técnica del lenguaje.
3. **Severance, C. R. (2016).** *Python para todos: Explorando la información con Python 3*. Colección de textos académicos en computación y lógica algorítmica.